

ENGG2112 Multi-Disciplinary Engineering

Semester 2, 2026
Week 5

Professor T. J. Lim



THE UNIVERSITY OF
SYDNEY

Celebrating 175 years

CRICOS 00026A TEQSA PRV12057



Motivation

Why Neural Networks in Engineering?

Applications of Neural Networks in Engineering

Structural Health Monitoring

Detecting cracks, fatigue and anomalies in bridges and aircraft from vibration and sensor signals

Automated Defect Detection

Spotting defects on production lines from images; faster and more consistent than manual inspection

Predictive Maintenance

Forecasting equipment failure from sensor data before it happens

Autonomous Control Systems

Learning control policies for robots, drones, and self-driving vehicles directly from data

What We're Covering Today

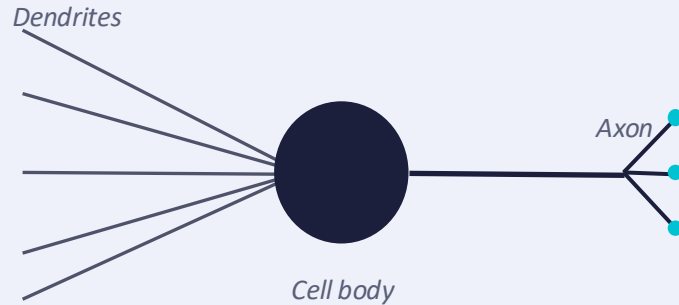


Multi-Layer Perceptrons

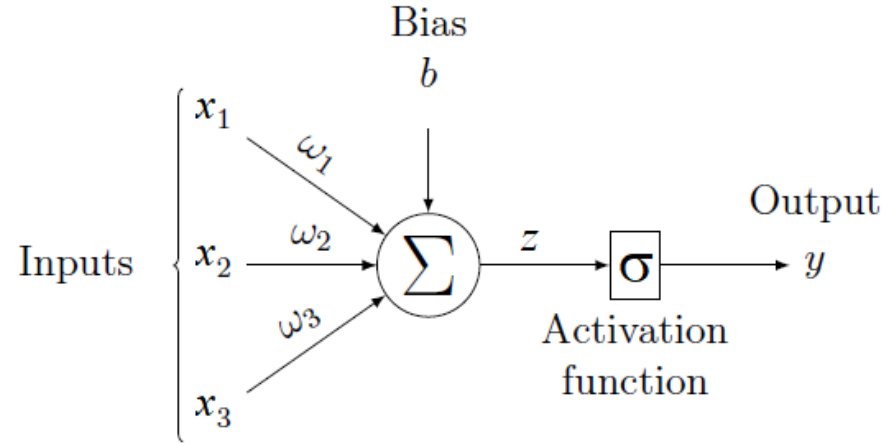
The Building Block

Borrowing from Biology: The Artificial Neuron

BIOLOGICAL NEURON



Signals arrive at the dendrites, are combined in the cell body, and — if strong enough — fire down the axon.



Inputs are scaled by weights, summed, and passed through an activation function to produce an output

The Perceptron

The full computation performed by a neuron:

$$z = \sum_{i=1}^K w_i x_i + b$$
$$\hat{y} = \sigma(z)$$

where $\sigma(\cdot)$ is known as an activation function. It is usually non-linear, and hence the perceptron performs a non-linear operation.

The weights $w_1, \dots, w_K$ and bias b are the parameters that need to be learned during training.

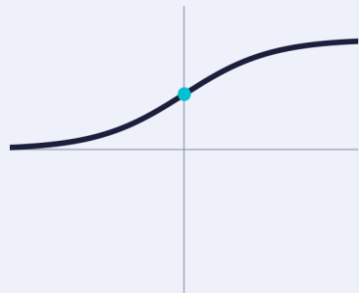
The neuron inputs $x_1, \dots, x_K$ are the input features in the first layer, and the outputs of a prior layer of neurons in the hidden layer.

The neuron output is labelled $\hat{y}$ but is only the output of the NN at the output layer; at other layers, the neuron output is an input to another layer of neurons.

Activation Functions

Three common choices:

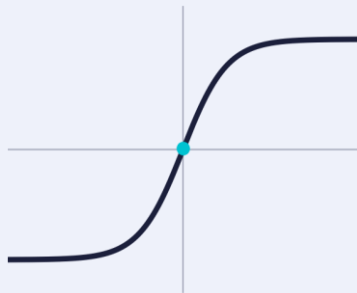
Sigmoid



$$\sigma(z) = 1 / (1 + e^{-z})$$

Squashes to (0, 1) — classic choice for output probabilities.

Tanh



$$\tanh(z)$$

Squashes to (-1, 1) — zero-centered, common in hidden layers.

ReLU



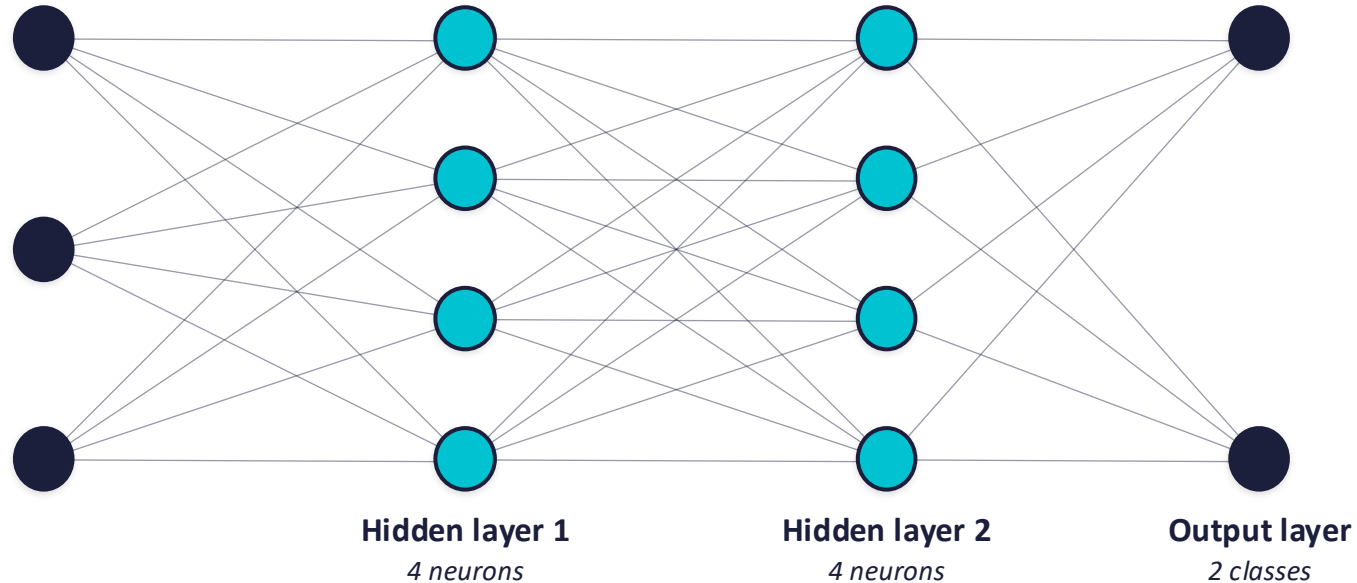
$$\max(0, z)$$

Cheap and effective — the default for hidden layers today.

Non-Linearity to Model Complex Systems

- Without the activation function, even a complex multi-layered system will be linear and cannot model cognitive systems or other complex systems.
- With activation functions to introduce non-linearity, increasing the complexity of the NN architecture allows us to successfully mimic cognitive functions such as image recognition.

Stacking Neurons: The Multilayer Perceptron



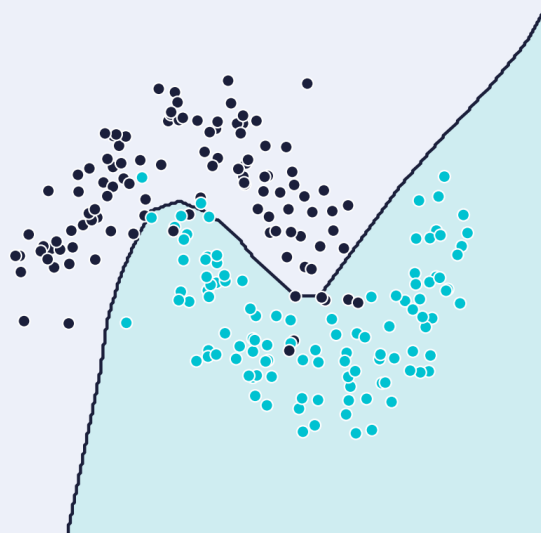
What is the Network Actually Learning?

Complex non-linear decision boundaries can be synthesized by an MLP.

The more layers and neurons, the more flexible the decision boundaries can be.

- Both blessing and curse – if training data is sufficient, MLP can solve very complex classification problems it has not seen before. But if training is insufficient, model may overfit and not generalize well.
- More on this later.

Multilayer perceptron — non-linear



Training and Back-Propagation

Minimizing loss function iteratively

Loss Functions

- In any supervised learning context, we want the ML model output $\hat{y}$ to be as close to the ground truth label y as possible.
 - But what does that mean precisely?
 - In classification, perhaps we can maximize $P[y = \hat{y}]$?
 - In regression, perhaps we should minimize $E[(y - \hat{y})^2]$?
- In both cases, we need the probability distribution of the features and the label.
 - Conceptually, we can learn this distribution to some degree using training data. Hence the more data we have, the better.
- Whether regression or classification, we need to define a loss function to minimize over the model parameters θ .
 - For regression, MSE (mean square error) is the natural choice.
 - For classification, the probability of error is not suitable but we have a proxy called cross entropy.

Mean Squared Error (MSE)

- In regression, the model directly outputs an estimate of the continuous label, i.e. $\hat{y} = f_{\theta}(x)$, as we saw earlier.
 - For a given x , $\hat{y}$ is a function of θ , hence $(y - \hat{y})^2$ will also be a function of θ .
 - Need to find the θ that minimizes $(y - \hat{y})^2$ on average, where averaging is over the distribution of x and y .
 - Since we only have training data but not exact distributions, we can approximate the desired average by averaging over all the training data, hence this loss function works:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- This is generally not a simple function of θ and hence we cannot minimize it in a closed-form expression.

Cross Entropy

- In a classification problem where y has a sample space with L values, the ML model must output a probability distribution over those L values.
 - We'll see how in just a while.
- In this case, we don't only have a single "point" estimate of y , as in regression.
 - Instead, the model outputs its confidence that y takes each of its L possible values, i.e. it outputs

$$p_1, \dots, p_L \quad \text{s.t.} \quad \sum_{l=1}^L p_l = 1$$

- We would naturally want $\max(p_1, \dots, p_L) = p_y$, where y is the ground truth label, for all training samples.
 - But this problem statement does not afford us any way to find the optimal θ .
- Hence, we need to bring in the concept of entropy.

Cross Entropy

- In general, the cross entropy of two distributions P and Q over the same discrete sample space $\mathcal{S}$ is

$$H(P, Q) = - \sum_{x \in \mathcal{S}} P(x) \log_2 Q(x)$$

- It is minimal when $P = Q$, i.e. the two distributions are identical. In that sense, it's a form of distance between two distributions.
- Now recall that our ML model outputs a probability distribution over the sample space of y , and that we have the ground truth value of y .
- Suppose $\mathcal{S}_y = \{1, 2, 3, 4\}$, and the true value of $y = 2$. Then the true distribution is $p_1 = p_3 = p_4 = 0$ and $p_2 = 1$.
- The ML model will estimate a different distribution from the true one, let's call this distribution $\hat{p}_l, l = 1, 2, 3, 4$.

Cross Entropy

- The cross entropy between the true distribution and the estimated one is thus

$$H(p, \hat{p}) = - \sum_{l=1}^4 p_l \log_2 \hat{p}_l = - \log_2 \hat{p}_y$$

- This is an incredibly elegant result! Since $\hat{p}_y$ is the y -th output of the ML model, its relationship to the parameter vector θ is explicit.
- For the i -th sample in the training set, we can compute $H(p^{(i)}, \hat{p}^{(i)})$, and so our loss function for classification can be the average of these cross-entropies, i.e.

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N H(p^{(i)}, \hat{p}^{(i)}) = -\frac{1}{N} \sum_{i=1}^N \log_2 \hat{p}_{y_i}$$

where y_i denotes the ground truth label of the i -th data sample.

Optimization of Loss Function

Training Using Gradient Descent

Method of Steepest Descent

- Both MSE and cross-entropy loss functions are differentiable with respect to θ , as we'll see soon. But still we can't solve

$$\nabla L(\theta) = \frac{dL(\theta)}{d\theta} = 0$$

- A simple numerical method for approximately solving this equation is to “descend” towards the nearest minimum through a sequence of steps taken in the direction of steepest descent.
 - The direction of steepest ascent is given by $\nabla L(\theta)$, hence the direction of steepest descent is the negative of that.
 - Denoting the step size or learning rate by μ , we have the SD iteration
$$\theta_{t+1} = \theta_t - \mu \nabla L(\theta_t)$$
 - The iterations continue until a “stopping criterion” is met, e.g. θ changes by less than a threshold from one iteration to the next, or a maximum number of iterations is reached.

Batch, Mini-Batch and Stochastic Gradient Descent (SGD)

- Recall that our two loss functions are averages over the entire set of training samples.
 - The size of this data set is normally very large (at least in the thousands).
- The computational complexity and time of gradient calculation both scale linearly with N , the size of the training data.
 - Using all training samples (known as **batch** processing) for gradient descent optimization will therefore consume a lot of resources and be very slow.
 - One idea is to use only a small **mini-batch** of samples per iteration, i.e. at the t -th iteration, we approximate the loss function by

$$L(\theta) \approx \frac{1}{N_b} \sum_{i=tN_b}^{(t+1)N_b-1} l_i(\theta)$$

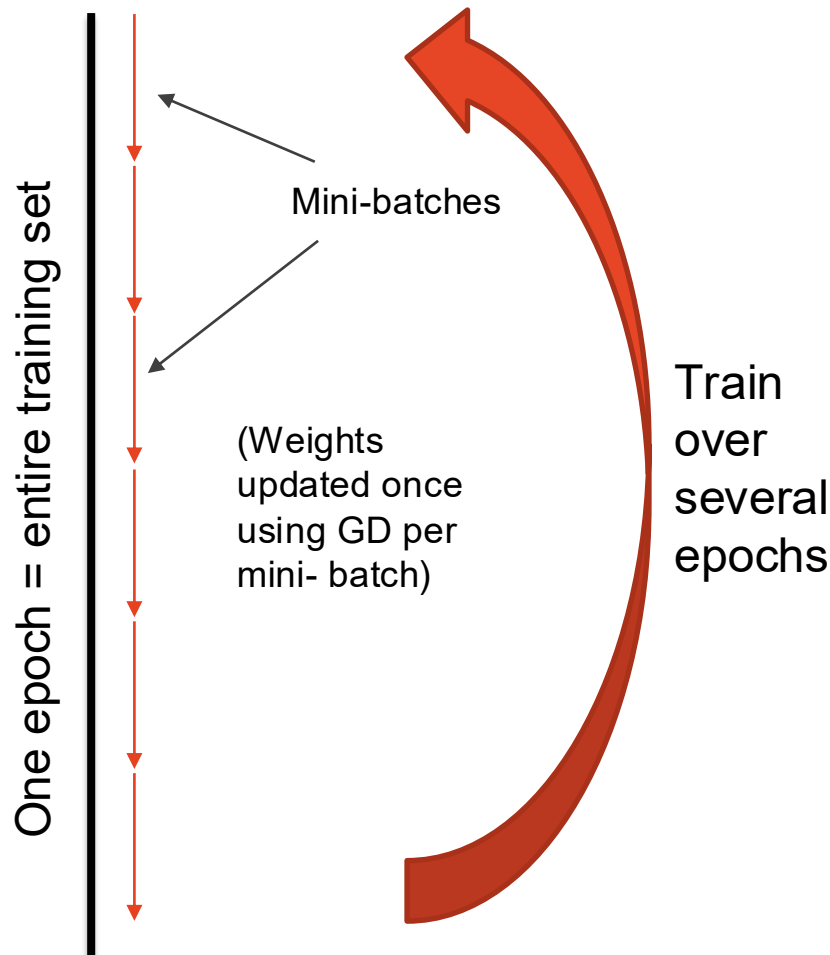
where $N_b \ll N$ is the mini-batch size, and $l_i(\theta)$ is either squared error or cross entropy for the i -th training sample.

Batch, Mini-Batch and Stochastic Gradient Descent (SGD)

- The mini-batch size N_b can be anything from 1 to N . Generally accepted terminology is:
 - Batch GD: $N_b = N$ (i.e. use all training samples per iteration)
 - Mini-Batch GD: $2 \leq N_b < N$ (i.e. use more than 1 but fewer than all training samples per iteration)
 - Stochastic GD: $N_b = 1$ (i.e. use only one sample per iteration)
- Unfortunately, SGD is also often used to refer to mini-batch GD. Putting aside terminology, the main idea here is:
 - Use N_b as a trade-off variable – smaller $N_b \Rightarrow$ noisier but faster and less complex updates.
 - Default in sklearn is $N_b = 200$. It can be reset using the `batch_size` input parameter.

Epochs and Batches

- As we proceed through the batches, we eventually pass all the training data through the SGD solver.
- Each complete pass through the training data is known as a training epoch.
- The number of epochs we use for training is arbitrary – it depends on how much time and computational resources we have.



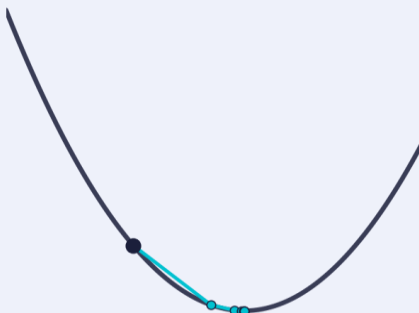
Choice of Step Size/Learning Rate

Too Small



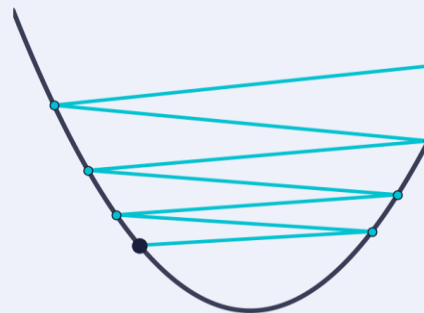
Crawls toward the minimum — training takes far longer than necessary.

Just Right



Converges quickly and smoothly toward the minimum.

Too Large

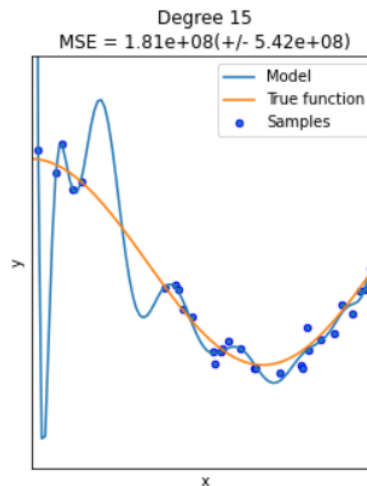
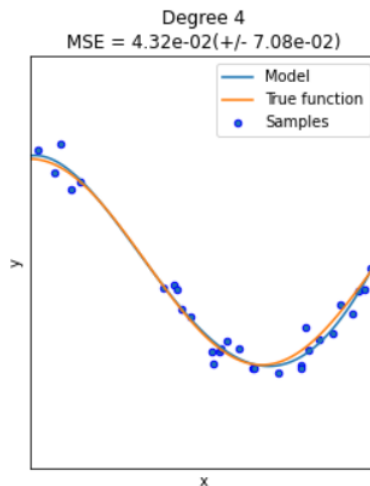
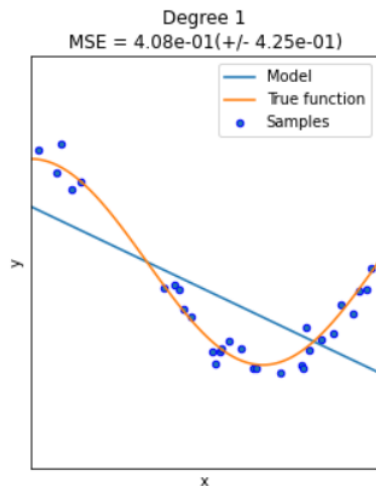


Overshoots the minimum — can oscillate or even diverge.

No need to pick step size manually. Optimizers like Adam adapt the step size automatically.

Overfitting and Regularization

- If we have more parameters than training data, e.g. we have an MLP having 1000 weights and 500 training samples, then we can drive the loss function to a value that matches every input feature perfectly with its class. This is an extreme case of the **overfitting** phenomenon.
 - Can visualize it as in the figures below.
 - Generalization performance will be poor.



Overfitting and Regularization

- Avoiding over-fitting by ensuring that number of training samples exceeds number of model parameters is obviously something we can do.
- However over-fitting is not necessarily due to such an easily detected scenario. It is usually because the underlying model is simpler than what we've assumed – but we don't know the underlying model!
 - All we can do is to try to drive the smallish weights (which are essentially redundant) down to zero.
 - The usual way to do this is to penalize the norm of the parameter vector, i.e. we add a term that drives the norm towards zero.

Backpropagation

- In SGD, we need to compute gradients of the loss function with respect to each and every one of the weights in the NN.
- To do this we use the chain rule of differentiation, starting from the output layer and progressing backwards.
- To illustrate, consider one of the output layer weights:

$$\hat{y} = \sigma(w_1x_1 + \dots) = \sigma(z)$$

- We use the Chain Rule to write

$$\frac{dL}{dw_1} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dw_1} = \frac{dL}{d\hat{y}} \sigma'(z)x_1$$

- All the terms on the RHS can be calculated.
- In the lecture, we will demonstrate finding the derivative with respect to one of the hidden layer weights.

Convolutional Neural Networks (CNNs)

Automatic Feature Extraction on 2D Inputs

Why Images Are Problematic for MLPs

- MLPs assume that features are just a flat list: if we swap two features in the order that they are fed into the MLP, the result will be unchanged.
 - In other words, adjacency carries no information as the ordering of features are arbitrary.
- If an image were to be fed into an MLP, we would "flatten" the pixels, i.e. turn the matrix of 2D pixel values into a 1D vector. This presents the first problem, complexity or "parameter explosion"
 - A 128 x 128 grayscale image turns into 16,384 features at the input layer.
 - If the first hidden layer only has 128 neurons, we already need over 2 million weights in that first layer.
- The second problem is that any detail (a face, a sign, etc.) when shifted even by one pixel will not be recognized as being the same thing.
 - The MLP has no built-in notion that something is the "same pattern but moved".

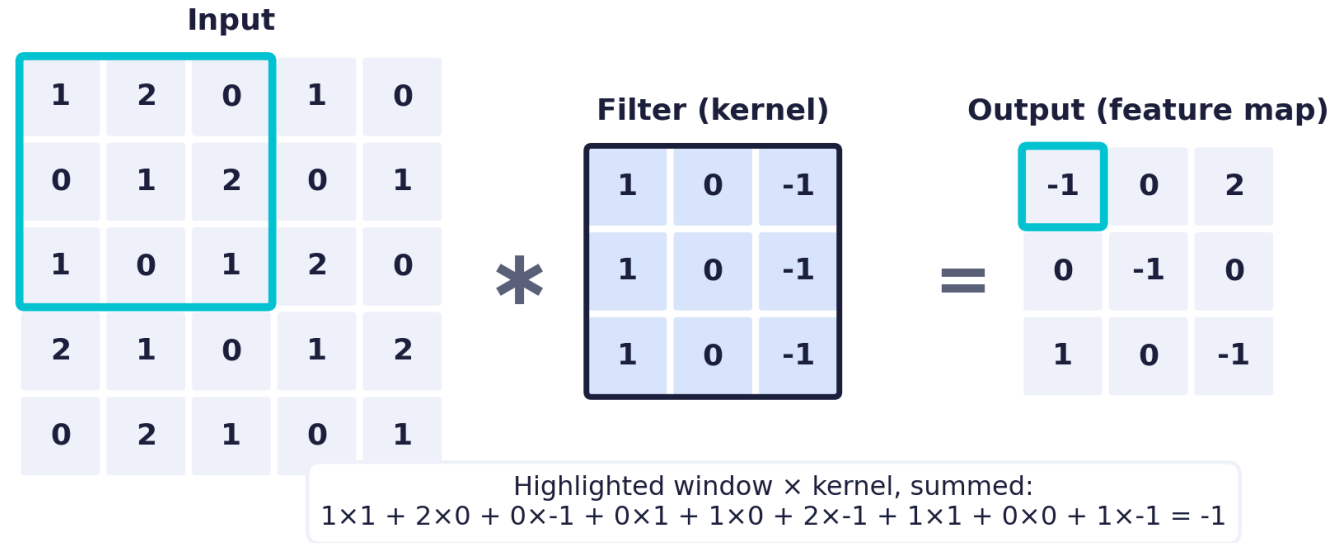
Convolution or Filtering to Reveal Features

- Convolution or linear filtering is nothing new. A filter for a 2D image can change the image so that certain colours are amplified or edges are sharpened – digital image processing has been doing this for years.
 - The effect of filtering depends critically on the filter kernel $w_{j,k}$, where the total number of weights in the kernel is user-defined. The filter output is

$$y_{m,n} = \sum_j \sum_k w_{j,k} x_{m-j,n-k}$$

- Depicted pictorially this is much less obtuse than this equation! See next page.

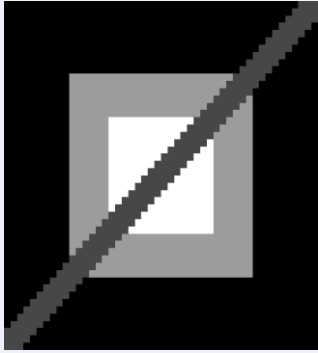
Convolution Illustration



This kernel is a vertical edge detector – if the image in the window has a vertical edge, say a value of 10 for the first two columns and 0 in the third column, what's the filter output?

Feature Maps: What a Filter Detects

Original



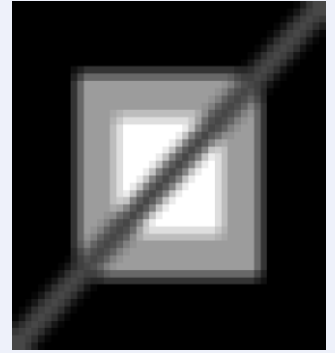
Vertical edges



Horizontal edges



Blur



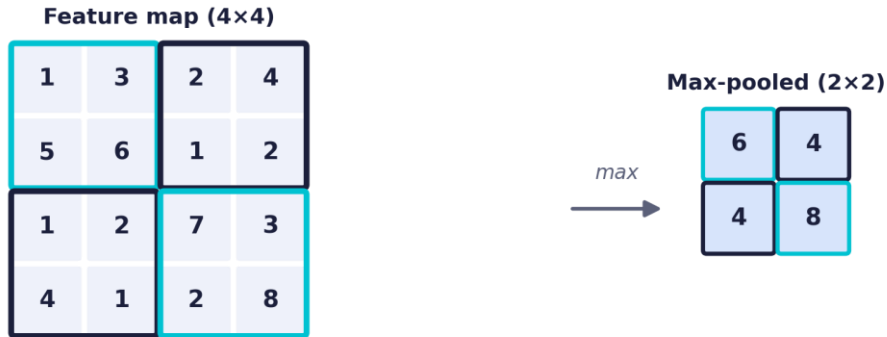
CNN Convolutional Layer

- A CNN starts with a convolutional layer where the kernels are not hand-designed but are learned from the data.
- In other words, instead of manually deciding what features to extract for a given image processing task, we let the learning algorithm decide from the data.
 - If the task is deciding whether a picture contains a furry animal or not, "furriness" could be a more important feature than "vertical edge".
- This automatic feature extraction step is the key to solving the 2nd problem of the MLP, its inability to recognise a feature that's been shifted.
- But it doesn't solve the complexity issue, which is where "max pooling" comes in.

CNN Pooling

- An image carries redundancy – an edge will carry over multiple pixels, so will a single colour or a texture.
 - This translates also into the feature map.
- It is therefore sufficient to collapse a small region of a feature map into just one number, e.g. within a 2 x 2 pixel square, retain only the largest value to represent the presence of that feature within the region.
- This operation is known as max-pooling, and solves the 1st MLP problem of parameter explosion.
- We can also have average-pooling, which retains the average of the feature map in a region.

Illustration of Max Pooling

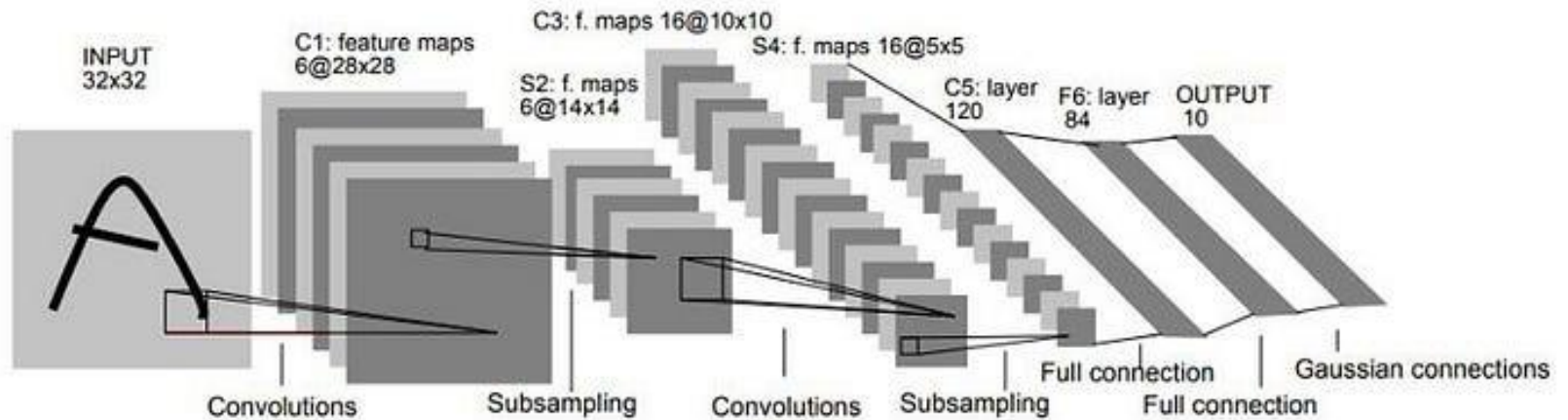


- We just reduced the number of inputs to the next layer by a factor of 4 without losing (much) information.
- Max pooling keeps only the strongest activation in each region, i.e. the clearest indication that the feature was found in that region.
 - That's all the information that the next layer needs for e.g. face recognition or anomaly detection.

LeNet-5: Pioneering CNN

Layer		Feature Map	Size	Kernel Size	Stride	Activation
Input	Image	1	32x32	-	-	-
1	Convolution	6	28x28	5x5	1	tanh
2	Average Pooling	6	14x14	2x2	2	tanh
3	Convolution	16	10x10	5x5	1	tanh
4	Average Pooling	16	5x5	2x2	2	tanh
5	Convolution	120	1x1	5x5	1	tanh
6	FC	-	84	-	-	tanh
Output	FC	-	10	-	-	softmax

LeNet-5: Pioneering CNN



Softmax Output Layer

Creating Probabilities Using Softmax

- Recall: in a classification problem where the label space has L values, an ML model should produce probability values $p_1, \dots, p_L$ where p_l quantifies the confidence that the model has that the true label is l .
- The forward propagation through a NN (or any ML model) is designed to result in L values, but there is no way that those L values will sum to 1 without normalization.
 - This normalization operation is known as softmax.
- There are obviously many ways in which we can transform L raw numbers into a probability distribution, but softmax makes the most sense:

$$\text{softmax}_i(z_1, \dots, z_L) = \frac{e^{z_i}}{\sum_{l=1}^L e^{z_l}}$$

- It allows the raw input to be any real value (negative or positive); the exponentiation magnifies the gap between the largest and smallest values; most importantly, it has a simple derivative with respect to any z_i .

Creating Probabilities Using Softmax

- Example 1: The task is classifying an image as “dog”, “cat” or “airplane”. If the raw NN outputs are (2.0, 1.0, 0.1), the softmax outputs are (0.659, 0.242, 0.099). Instead of a factor of two between the top two choices, now the ratio is 2.72.
- Example 2: Same task as above. If the raw NN outputs are (1, 2, 8), the model is more confident in the most likely label than above. The softmax function accentuates that certainty and outputs probabilities of (0.001, 0.002, 0.997).

Recap

- Neural Networks attempt to mimic the structure of the brain
- Basic building block is the neuron or perceptron
- Most straightforward NN is the multi-layer perceptron (MLP), which is a feedforward NN
- Described MSE and cross-entropy loss functions and how to minimize them using training data and SGD
- Described Convolutional NN as an ML model naturally able to handle 2D (image) inputs.
- Ended with a discussion of the softmax function.

See you next week!



Survey on this lecture